WOXSEN UNIVERSITY

School of Technology

BLOCKCHAIN TECHNOLOGY

MINI-PROJECT REPORT

BlockProof

Blockchain-Based File Integrity
Verification System

B.Tech Computer Science and Engineering

Specialization in Blockchain, IoT and Cybersecurity

Semester V | Academic Year 2026–2027

Student Name	Rishvin Reddy
Roll Number	24WU0103006
Section	BIC - TIGERS
Programme	B.Tech Computer Science and Engineering
Specialization	Blockchain, IoT and Cybersecurity
Subject	Blockchain Technology Laboratory
Faculty	Dr. Rajkumar Kalimuthu
Designation	Associate Professor & Head of the Department (HoD)
Institution	Woxsen University, School of Technology

Hyderabad, Telangana

12 October 2026

Certificate

This is to certify that **Rishvin Reddy**, a student of B.Tech Computer Science and Engineering with specialization in Blockchain, IoT and Cybersecurity, Semester V, has prepared this mini-project report titled **"BlockProof: Blockchain-Based File Integrity Verification System"** for the Blockchain Technology Laboratory during the academic year 2026–2027.

The project integrates SHA-256 hashing, a Solidity smart contract, and Web3.js-based interaction with a local Ethereum-compatible development network. The report describes the design, implementation, test strategy, and observed algorithm-level verification results.

Student Signature	**Faculty Signature**
Rishvin Reddy	**Dr. Rajkumar Kalimuthu** Associate Professor & Head of the Department (HoD)

Place: Woxsen University, Hyderabad

Declaration

I, **Rishvin Reddy**, declare that this report describes the design and implementation of BlockProof, a mini-project prepared for the Blockchain Technology Laboratory in Semester V of the academic year 2026–2027. The report identifies the implemented source files, the test cases included in the project, and the scope and limitations of the results.

The application is designed to calculate SHA-256 digests of uploaded files, register digests on a local blockchain through a Solidity smart contract, and verify subsequent files against the registry. The file contents are not stored on-chain.

Student Name: Rishvin Reddy

Signature: _____

Date: 12 October 2026

Acknowledgement

I would like to express my sincere gratitude to **Associate Professor & Head of the Department (HoD) Dr. Rajkumar Kalimuthu** and the School of Technology, Woxsen University, for the academic guidance and laboratory context that informed this mini-project. The Blockchain Technology Laboratory experiments provided the foundation for combining cryptographic hashing, smart contracts, and application-level blockchain interaction in a single demonstrable system.

I also acknowledge the maintainers of Node.js, Solidity, Web3.js, and Hardhat, whose tools make local blockchain experimentation and reproducible development possible.

Abstract

BlockProof is a blockchain-based file integrity verification system developed as a mini-project for the Blockchain Technology Laboratory. It addresses a focused problem: determining whether a file presented later has the same byte content as a file previously registered. The system computes a SHA-256 digest using Node.js, records that digest with a registering address and timestamp in a Solidity smart contract, and uses Web3.js to communicate with a local Ethereum-compatible development network. A lightweight web interface supports registration, verification, and recent-record inspection.

The file itself is not placed on the blockchain. Only its fixed-length digest and limited registration metadata are recorded, reducing data exposure and avoiding the cost and impracticality of storing complete documents on-chain. A matching digest indicates that the submitted bytes match the registered digest, subject to the security assumptions of SHA-256 and the trustworthiness of the original registration. A mismatch means the digest is not present in the registry; it does not, by itself, establish malicious modification.

The deliverable includes the Solidity contract, Node.js server, web interface, deployment script, automated Node.js tests, and setup documentation. The included automated tests cover digest formatting, repeatability, changed-content detection, empty-file hashing, and static contract guardrails. End-to-end blockchain execution must be run in the target environment using the included local-chain instructions before claiming a live deployment result.

Contents

1 Introduction **1**

2 Problem Statement **1**

3 Objectives **1**

4 Scope and Assumptions **1**

5 Tools and Technologies **2**

6 System Architecture **2**

6.1 Data flow 3

7 Design and Implementation **3**

7.1 SHA-256 hashing 3

7.2 Solidity registry 3

7.3 Web3.js integration 4

7.4 Web interface 4

8 Security and Privacy Considerations **4**

9 Testing Methodology **5**

10 Algorithm-Level Results **5**

11 Demonstration Procedure **6**

12 Limitations and Future Enhancements **6**

13 Conclusion **6**

Introduction

Digital documents are routinely shared between people and systems. A recipient may need to check whether a file is identical to a previously recorded version. File names, file sizes, and timestamps are not sufficient to establish byte-for-byte equality. Cryptographic hash functions provide a compact fingerprint of file contents, while a blockchain can provide a shared, append-only reference to a fingerprint.

BlockProof combines these ideas in a small educational application. It uses SHA-256 to fingerprint file bytes, a Solidity contract to store the fingerprint, and Web3.js to invoke the contract through JSON-RPC. The implementation targets a local development chain so the complete workflow can be demonstrated without deploying to a public network or spending real cryptocurrency.

Problem Statement

A file may be altered after it is registered or shared. Without a trusted reference to the original content, a user may not be able to determine whether the current bytes match the earlier version. The project therefore needs a simple mechanism to create a cryptographic reference, store it in a tamper-resistant registry, and compare later submissions with that reference.

Objectives

1. Generate SHA-256 digests from uploaded file bytes.
2. Store digests and registration metadata in a Solidity smart contract.
3. Integrate the application with the contract through Web3.js.
4. Provide a browser interface for registration, verification, and record inspection.
5. Test repeatability, content-change detection, and key contract-source guardrails.
6. Keep document contents off-chain and document the system's limits.

Scope and Assumptions

The project is a local educational prototype. It supports file uploads up to 10 MB, computes the digest server-side, and submits transactions to a local Hardhat network. The demo expects the local JSON-RPC node and deployment metadata to be available. It does not provide user accounts, production key management, public-chain deployment, file recovery, malware scanning, authorship proof, or a claim that the first registered version is truthful. The local development chain is intended only for testing.

Tools and Technologies

Table 1: Tools used in the implementation

Tool	Role	Use in BlockProof
Node.js	Server runtime	Handles uploads and computes SHA-256 using the built-in `crypto` module.
Solidity 0.8.24	Smart contract language	Stores file digests and registration metadata.
Web3.js 4.x	Blockchain client	Calls contract methods using the local JSON-RPC endpoint.
Hardhat 2.x	Local development environment	Runs a local EVM-compatible chain and deploys the contract.
Express	HTTP server	Serves the interface and exposes API endpoints.
Multer	Multipart upload handling	Holds uploads in memory with a 10 MB limit.
HTML, CSS, JavaScript	User interface	Provides registration, verification, and ledger views.

System Architecture

Figure 1 shows the flow between the browser, server, hashing component, Web3.js client, and smart contract. Uploaded bytes are processed in server memory. Only the digest is sent to the contract.

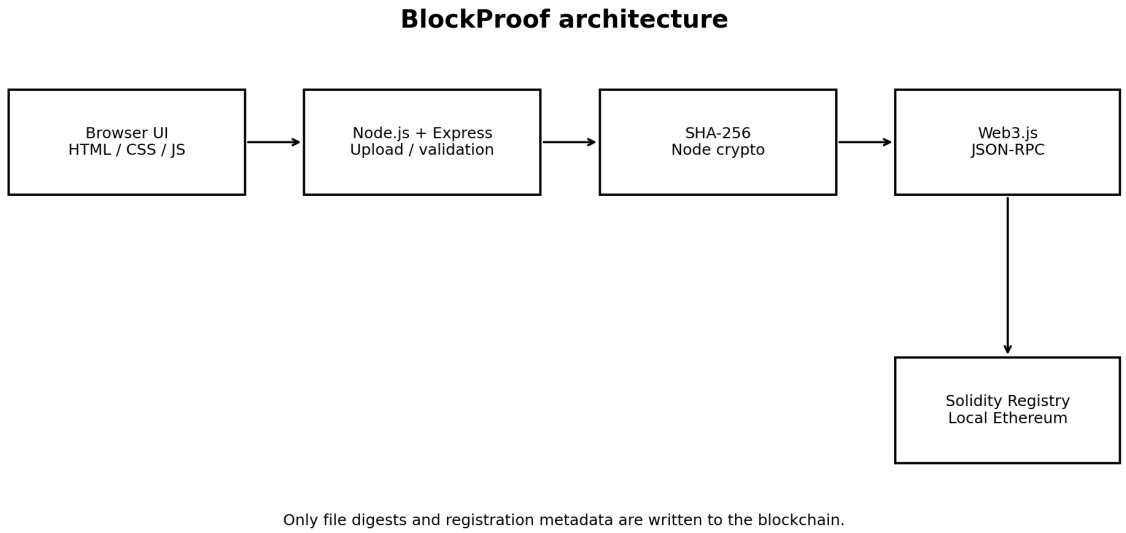


Figure 1: BlockProof system architecture

Data flow

1. The user selects a file in the browser.
2. The server receives the file in memory and calculates SHA-256 over its raw bytes.
3. For registration, the server converts the 64-character hexadecimal digest to a `bytes32` value and sends a transaction to the contract.
4. The contract stores the digest, sender address, timestamp, and existence flag, and emits a registration event.
5. For verification, the server recalculates the digest and queries the contract.
6. The interface displays whether a record exists for the digest.

Design and Implementation

SHA-256 hashing

The server uses Node.js `crypto.createHash('sha256')` to compute the digest from a file buffer. A SHA-256 digest is 256 bits, conventionally represented as 64 hexadecimal characters. Hashing file bytes rather than the file name ensures that renaming an unchanged file does not change the digest, while changing its content ordinarily produces a different digest.

Listing 1: Core SHA-256 operation

```
function sha256(buffer) {
  return crypto.createHash("sha256").update(buffer).digest("hex");
}
```

Solidity registry

The contract stores each digest in a mapping from `bytes32` to a `FileRecord`. The record contains the digest, registering address, block timestamp, and existence flag. A separate array preserves registration order for recent-record display. The contract rejects a zero digest and duplicate registrations and emits a `FileRegistered` event.

Listing 2: Core contract record structure

```
struct FileRecord {
    bytes32 fileHash;
    address registeredBy;
    uint256 timestamp;
    bool exists;
}
```

Web3.js integration

The Express server constructs a Web3 contract instance from the compiled ABI and the deployment address stored in `deployment/local.json`. Registration calls `registerFile`; verification calls `getRecord`; the ledger view uses `getRecordCount` and `getRecordAt`. The local development node exposes unlocked test accounts, so the server uses the first account returned by the local RPC node. This pattern is restricted to local development and must not be used as production key management.

Web interface

The interface has three areas: file registration, file verification, and recent registrations. It presents the digest, file size, transaction hash, block number, registration address, and verification outcome where available. The design preview in Figure 2 illustrates the interface layout. It is a generated visual preview of the authored UI, not evidence of a live browser session.

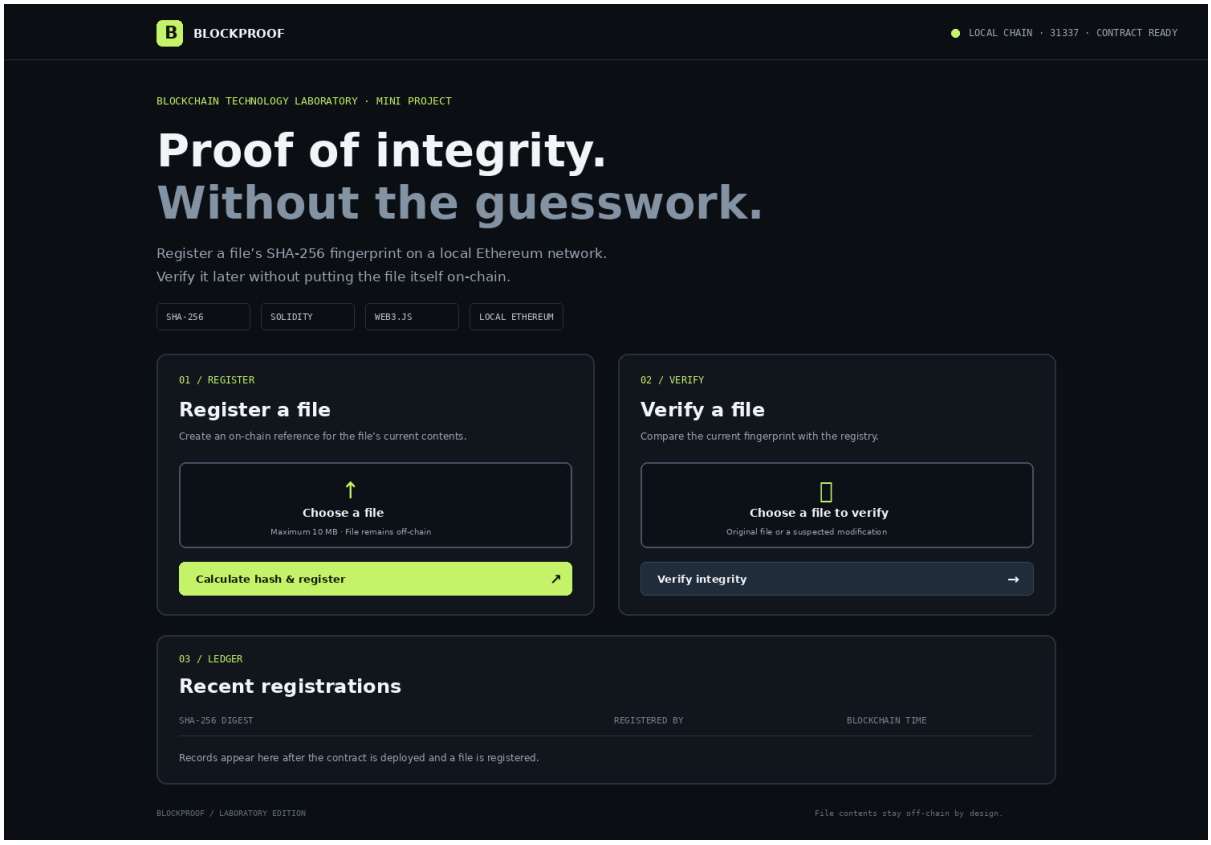


Figure 2: BlockProof interface design preview

Security and Privacy Considerations

- **Off-chain file contents:** The application does not persist uploaded file contents. Uploads are held in memory for processing.
- **Input limits:** Multer limits file size to 10 MB and accepts one file per request.

- **Hash validation:** The server validates that a digest passed to the contract is exactly 64 hexadecimal characters before converting it to `bytes32`.
- **Duplicate handling:** The contract rejects duplicate registrations for the same digest.
- **Local-chain boundary:** The Hardhat node and its pre-funded accounts are for local testing only. The RPC service must not be exposed to untrusted networks.
- **Interpretation:** A match means the current bytes have a digest registered on-chain. It does not prove authorship, legal ownership, or that the original registration was honest.
- **Metadata privacy:** Registering addresses and timestamps are public on the configured chain. No personal document metadata is intentionally stored in the contract.

Testing Methodology

The project includes a Node.js built-in test suite in `test/`. The hash tests validate digest shape, repeatability, changed-content detection, and the known SHA-256 digest for empty input. Static contract tests check the presence of the required public methods, zero/duplicate guards, and registration event declaration. These tests are useful baseline checks but do not prove that an EVM transaction has been mined or that the browser-to-chain flow has been executed in a live environment. The included Node.js suite was executed with Node.js v22.16.0 using `node --test test/*.test.js`; all 7 tests passed (0 failures). The suite consists of unit-level hashing checks and static source checks, not an end-to-end EVM test.

Table 2: Included automated test cases

ID	Test	Expected condition
T1	Digest format	Output contains 64 lowercase hexadecimal characters.
T2	Repeatability	Identical byte sequences produce identical digests.
T3	Modified content	A one-byte/content change produces a different digest.
T4	Empty file	Digest equals the standard SHA-256 empty-input value.
T5	Contract methods	Registration, record lookup, count, and indexed lookup are present.
T6	Guardrails and event	Zero and duplicate hashes are rejected; registration event is declared.

Algorithm-Level Results

The following deterministic checks were independently reproduced using Python's standard SHA-256 implementation while preparing the report. They demonstrate the hash behavior, not a live blockchain transaction.

Table 3: Observed SHA-256 behavior

Input / check	Observed result
Empty byte sequence	`e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934c`
Same bytes hashed twice	Digests equal
Different content (`document-v1` vs. `document-v2`)	Digests differ
Digest representation	64 hexadecimal characters

Interpretation. The digest tests support the expected content-fingerprinting behavior. The project source also includes a local-chain deployment and application workflow. A live EVM integration result is not claimed in this report because the target environment's dependency installation, contract compilation, and local RPC execution must be verified when the project is run.

Demonstration Procedure

1. Start the Hardhat local node with `npm run chain`.
2. In a second terminal, run `npm install`, `npm run compile`, and `npm run deploy:local`.
3. In a third terminal, run `npm start` and open http://localhost:3000.
4. Create a text file containing `BlockProof demo version 1` and register it.
5. Verify the unchanged file; the expected outcome is **MATCH**.
6. Change the content to `BlockProof demo version 2` and verify again; the expected outcome is **NO MATCHING RECORD**.
7. Inspect the transaction hash and recent-record list to confirm the on-chain registration in the running local environment.

Limitations and Future Enhancements

The prototype uses a local development chain and a development account exposed by the local RPC node. It is not designed for production deployment. Future work could add authenticated users, role-based registration, an independently deployed public testnet contract, transaction receipt polling, exportable verification reports, encrypted off-chain storage, and an EVM integration test that automatically deploys and exercises the contract. Such enhancements should preserve the principle that private file contents are not written directly to the blockchain.

Conclusion

BlockProof brings together three core laboratory concepts: SHA-256 hashing, Solidity smart contracts, and Web3.js interaction. It defines an end-to-end workflow for registering a file digest and checking whether later bytes match a previously registered digest. The implementation keeps file contents off-chain, validates the digest representation, handles duplicate registrations

in the contract, and provides a web interface for the primary operations.

The automated tests included with the source cover deterministic hashing behavior and key contract-source checks. The remaining verification step is to install dependencies and execute the local Hardhat workflow in the intended environment to confirm compilation, deployment, transaction submission, and browser integration. The project therefore provides a complete, reproducible implementation package without misrepresenting a live-chain run as already observed.

References

1. National Institute of Standards and Technology, *Secure Hash Standard (SHS)*, FIPS PUB 180-4, 2015. https://doi.org/10.6028/NIST.FIPS.180-4
2. Solidity documentation, https://docs.soliditylang.org/
3. Web3.js documentation, https://docs.web3js.org/
4. Hardhat documentation, https://hardhat.org/docs
5. Node.js Crypto documentation, https://nodejs.org/api/crypto.html